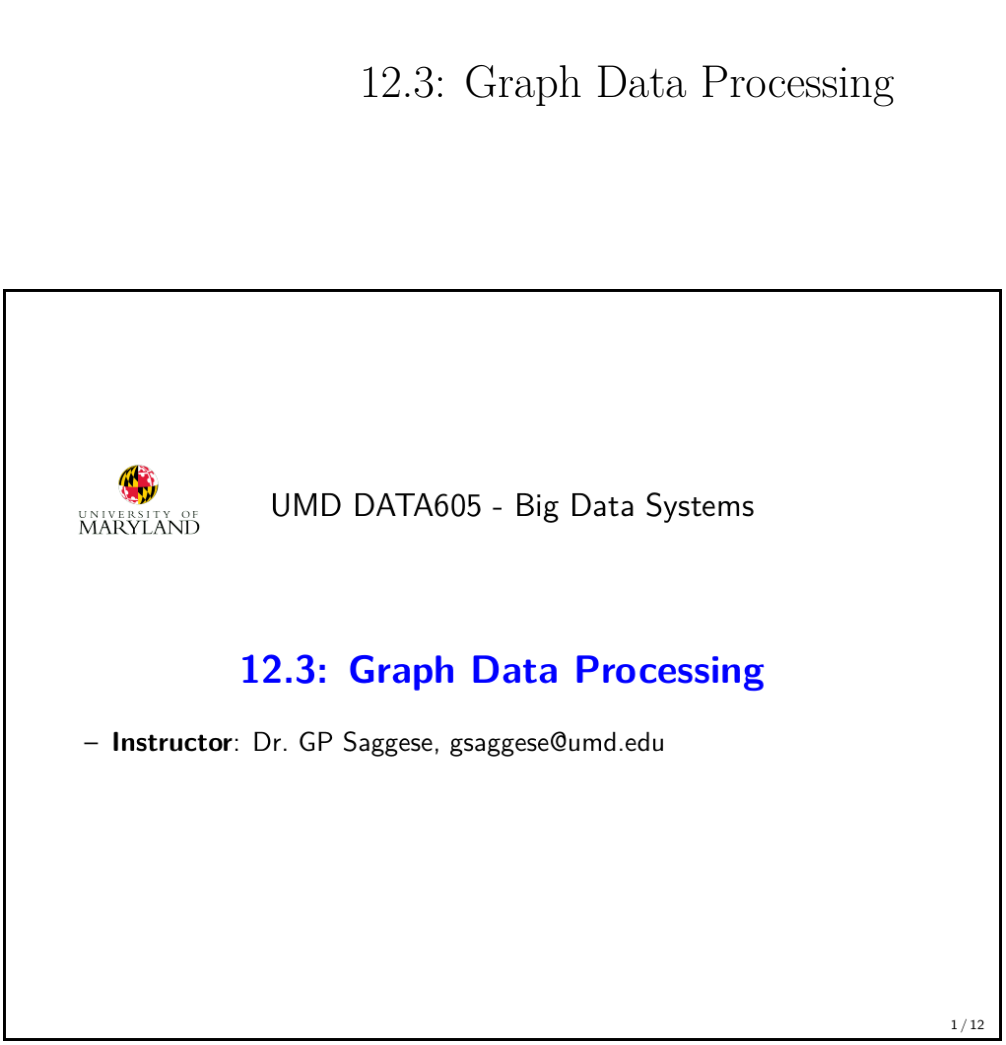
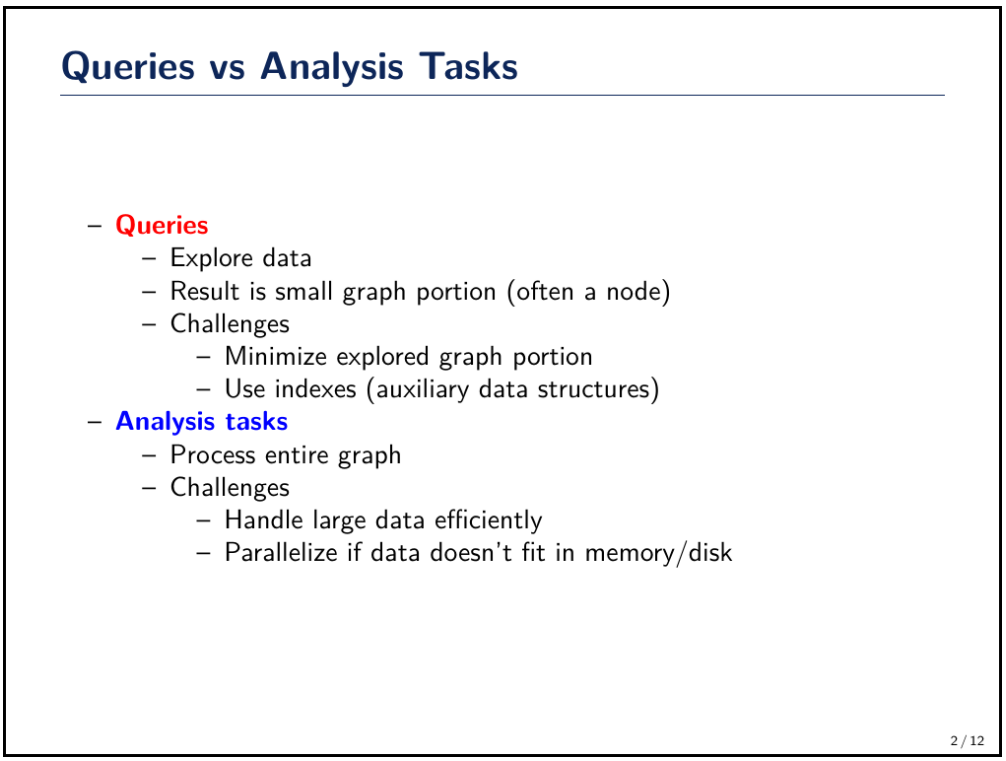
12.3: Graph Data Processing



2 / 12: Queries vs Analysis Tasks



– **Queries vs Analysis Tasks**

– **@Queries@**

  – **Explore data**: Queries are used to look into specific parts of a dataset. Think of it like asking a question to get a specific answer from the data.
  – **Result is small graph portion (often a node)**: When you run a query, you usually get a small piece of information back, like a single point or node in a larger network or graph.
  – **Challenges**
    – **Minimize explored graph portion**: The goal is to look at only the necessary parts of the data to get your answer, which saves time and resources.
    – **Use indexes (auxiliary data structures)**: Indexes help you find information quickly, like a book index helps you find topics without reading every page.

– **@Analysis tasks@**

  – **Process entire graph**: Unlike queries, analysis tasks involve looking at the whole dataset to understand it better or find patterns.
  – **Challenges**
    – **Handle large data efficiently**: Big datasets can be hard to work with, so it's important to find ways to process them without wasting time or resources.
    – **Parallelize if data doesn't fit in memory/disk**: If the data is too big to handle all at once, breaking it into smaller parts and processing them simultaneously can help. This is called parallelization, and it makes the task more manageable.

3 / 12: Graph Algorithms

Graph Algorithms

- **Graph algorithms** are general algorithms applicable to various graph types
  - Network flows (e.g., max flow, min cut)
  - Spanning trees (e.g., minimal connectivity)
  - ...
- **Applications**
  - Logistics and supply chain optimization
  - Electric grid design
  - Telecommunications (e.g., bandwidth allocation)

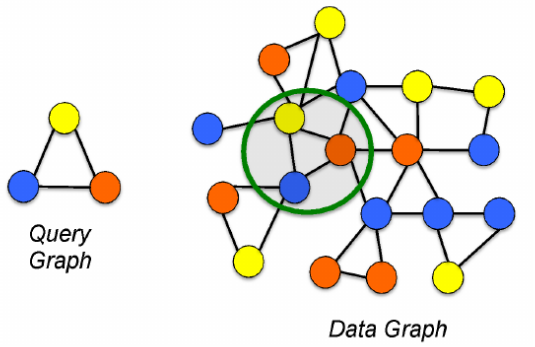
3 / 12

- **Graph Algorithms**
  - **Graph algorithms** are a set of procedures used to solve problems related to graphs, which are structures made up of nodes (or vertices) connected by edges. These algorithms are versatile and can be applied to different types of graphs, whether they are directed, undirected, weighted, or unweighted.
    - **Network flows**: These algorithms deal with the movement of resources through a network. For example, the *max flow* algorithm finds the maximum possible flow in a network from a source to a sink, while the *min cut* algorithm identifies the smallest set of edges that, if removed, would disconnect the source from the sink.
    - **Spanning trees**: These algorithms focus on connecting all nodes in a graph with the minimum number of edges. A common example is the *minimal spanning tree*, which ensures all nodes are connected with the least total edge weight, useful for minimizing costs.
- **Applications**
  - **Logistics and supply chain optimization**: Graph algorithms help in planning efficient routes and managing resources in supply chains, ensuring goods are delivered in the most cost-effective manner.
  - **Electric grid design**: They are used to design power distribution networks that minimize energy loss and ensure reliable electricity supply.
  - **Telecommunications**: In this field, graph algorithms assist in optimizing bandwidth allocation and designing networks that can handle data efficiently, ensuring smooth communication services.

4 / 12: Subgraph Matching

Subgraph Matching

- **Subgraph Matching** = find matching instances of a small pattern in a larger graph
  - Patterns are typically small and fixed
  - Matching can be *approximate*
- **Applications**
  - Fraud detection (e.g., detecting fraudulent transaction patterns)
  - Bioinformatics (e.g., finding protein interaction motifs)
  - Social network analysis (e.g., community pattern detection)



4 / 12

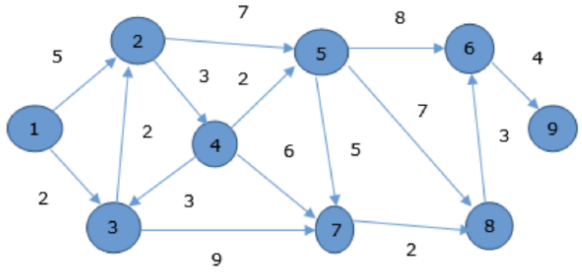
- **Subgraph Matching**: This concept involves identifying instances of a smaller pattern, known as a subgraph, within a larger graph.
  - **Patterns**: These are typically small and fixed, meaning they have a specific structure that we are trying to find within the larger graph.
  - **Approximate Matching**: Sometimes, the match does not need to be exact. Approximate matching allows for some flexibility, which can be useful in real-world applications where data might be noisy or incomplete.
- **Applications**: Subgraph matching is used in various fields due to its ability to identify specific patterns within complex networks.
  - **Fraud Detection**: By identifying patterns that are indicative of fraudulent activities, such as unusual transaction sequences, subgraph matching can help in detecting fraud.
  - **Bioinformatics**: In this field, subgraph matching can be used to find specific protein interaction motifs, which are crucial for understanding biological processes.
  - **Social Network Analysis**: It helps in detecting community patterns, which can provide insights into how information spreads or how groups are formed within social networks.

The accompanying image illustrates a query graph (a small pattern) and a data graph (a larger network), showing how the query graph is identified within the data graph.

5 / 12: Shortest Path Queries

Shortest Path Queries

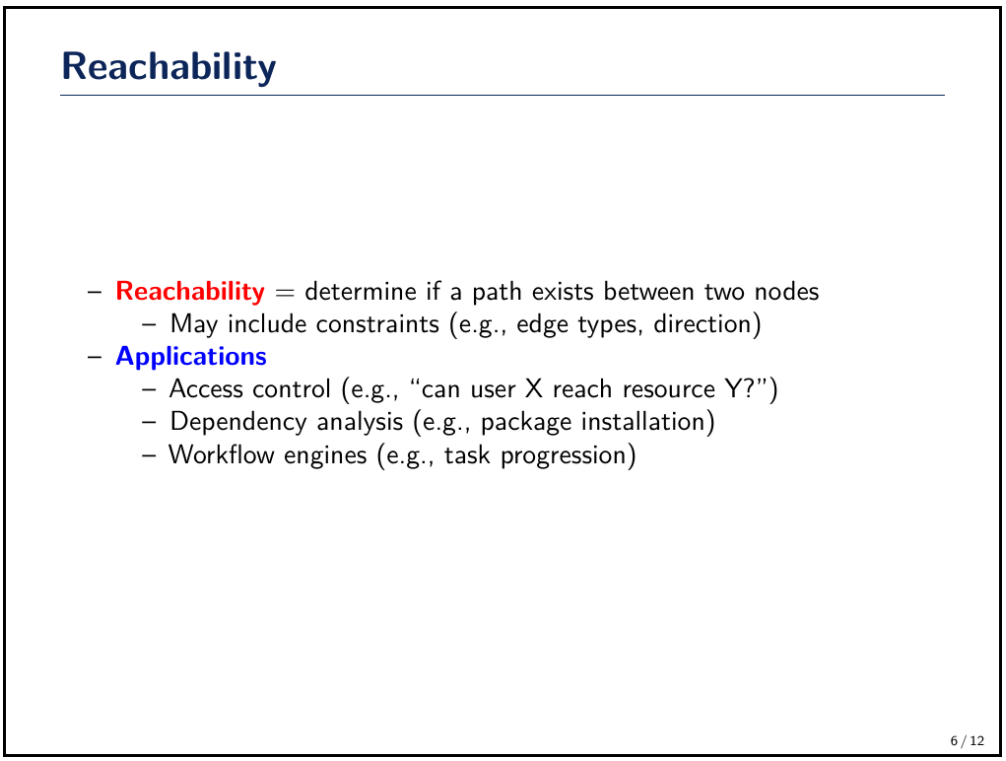
- **Shortest Path Queries** = find the shortest or optimal path between two nodes
  - May consider edge weights (e.g., distance, cost)
- **Applications**
  - GPS navigation (e.g., Google Maps)
  - Network routing (e.g., Internet traffic routing)
  - Robotics (e.g., path planning)



5 / 12

- **Shortest Path Queries**
  - *Shortest Path Queries* involve finding the shortest or most efficient path between two nodes in a graph.
  - These queries often take into account edge weights, which can represent various factors like distance or cost.
- **Applications**
  - **GPS Navigation**: Systems like Google Maps use shortest path algorithms to provide users with the quickest route to their destination.
  - **Network Routing**: Internet traffic is directed efficiently using shortest path calculations to minimize latency and congestion.
  - **Robotics**: Robots use path planning to navigate environments, ensuring they take the most efficient route to complete tasks.
- **Graph Illustration**
  - The image shows a directed graph with nodes and weighted edges.
  - Each edge has a number representing its weight, which could be a distance or cost.
  - The goal is to determine the shortest path from one node to another, considering these weights.

6 / 12: Reachability

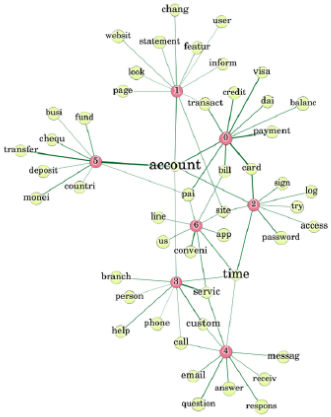


- **Reachability**
  - *Reachability* is about figuring out if there is a way to get from one point (node) to another in a network or graph. Think of it like checking if there's a route from your house to a friend's house on a map.
  - Sometimes, there are rules or limits on how you can travel between points. For example, you might only be able to use certain roads (edge types) or travel in one direction (direction).
- **Applications**
  - *Access control* uses reachability to check if a user can access a particular resource. For instance, it answers questions like "Can user X access file Y?" by determining if there's a valid path from the user to the resource.
  - *Dependency analysis* involves understanding how different parts of a system depend on each other. For example, when installing software, it checks if all necessary components (packages) can be reached and installed in the right order.
  - *Workflow engines* use reachability to manage tasks. They ensure that tasks are completed in the correct sequence, like making sure step A is done before moving on to step B in a project.

7 / 12: Keyword Search

Keyword Search

- **Keyword Search** = find smallest subgraph containing all specified keywords
- **Applications**
  - Knowledge graphs (e.g., question answering)
  - Enterprise search (e.g., documents linked by shared entities)
  - Academic citation networks



7 / 12

- **Keyword Search**: This concept involves identifying the smallest subgraph within a larger graph that contains all specified keywords. Essentially, it's about finding the most concise way to connect all the relevant terms or nodes in a network. This is particularly useful in large datasets where you want to focus on specific information without getting lost in irrelevant data.
- **Applications**:
  - **Knowledge Graphs**: These are used in question answering systems where the goal is to find connections between different pieces of information. By using keyword search, systems can efficiently locate the necessary data to answer queries.
  - **Enterprise Search**: In a business context, keyword search helps in finding documents or information linked by shared entities, such as common topics or people, making it easier to retrieve relevant information quickly.
  - **Academic Citation Networks**: Researchers can use keyword search to find papers or articles that are interconnected through citations, helping them to trace the development of ideas or theories over time.

The accompanying image likely illustrates a network graph where nodes represent keywords or entities, and edges show the relationships between them. This visual representation helps in understanding how different terms are interconnected within the dataset.

8 / 12: Historical Queries

Historical Queries

- **Historical Queries** = find nodes with similar evolution or temporal behavior
  - Typically on dynamic or time-stamped graphs
- **Applications**
  - Stock market analysis (e.g., similar price movement)
  - Social media trends (e.g., user behavior over time)
  - Epidemiology (e.g., disease spread patterns)

8 / 12

- **Historical Queries**
  - **@Historical Queries@**: This concept involves identifying nodes within a graph that exhibit similar patterns or changes over time. These nodes are part of dynamic or time-stamped graphs, which means the data they represent changes as time progresses. By analyzing these patterns, we can gain insights into how certain entities evolve or behave over a period.
  - **Applications**:
    - **Stock market analysis**: In this context, historical queries can be used to find stocks that have similar price movements over time. This can help investors identify trends or predict future behavior based on past performance.
    - **Social media trends**: By examining user behavior over time, historical queries can help identify trends in social media usage, such as how certain topics gain popularity or how user engagement changes.
    - **Epidemiology**: In the study of disease spread, historical queries can be used to track and predict patterns of disease transmission, helping public health officials understand and respond to outbreaks more effectively.

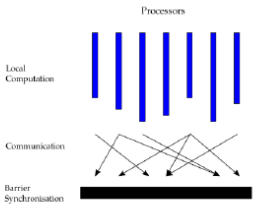
Graph Data Processing Systems

- This section likely introduces systems designed to handle and analyze graph data, which is crucial for performing historical queries efficiently. These systems are optimized to manage the complex relationships and temporal data inherent in dynamic graphs.

9 / 12: Bulk Synchronous Parallel Model

Bulk Synchronous Parallel Model

- BSP is a model for parallel computation using synchronized steps
- **Computation is organized into supersteps**
  - *Local computation*
  - *Message passing*
    - Parallel processors to exchange data
    - Messages are sent during a superstep but delivered in the next
  - *Synchronization barriers*
    - Ensure all processors complete a superstep before moving on



- **Pros**
  - Deterministic and predictable behavior
  - Simple abstraction for reasoning about parallelism
  - Suitable for graph computations with regular communication patterns
- **Cons**
  - Global synchronization can cause idle time (stragglers)

9 / 12

- **Bulk Synchronous Parallel (BSP) Model**
  - BSP is a framework for parallel computing that organizes tasks into synchronized steps called supersteps. This model helps manage the complexity of parallel computation by breaking it down into manageable parts.
- **Computation is organized into supersteps**
  - *Local computation*: Each processor performs its own calculations independently during a superstep.
  - *Message passing*: Processors exchange data with each other. Messages are sent during one superstep and received in the next, allowing for coordination and data sharing.
  - *Synchronization barriers*: These ensure that all processors complete their tasks for a superstep before any move on to the next. This coordination is crucial for maintaining order and consistency in computations.
- **Pros**
  - The BSP model offers deterministic and predictable behavior, making it easier to understand and manage parallel tasks.
  - It provides a simple abstraction that helps in reasoning about parallelism, especially useful in graph computations where communication patterns are regular.
- **Cons**
  - The need for global synchronization can lead to idle time, especially if some processors finish their tasks earlier than others (known as stragglers).
  - It may not be efficient for workloads that are highly irregular or require dynamic computation patterns, as the synchronization can become a bottleneck.

10 / 12: Pregel

Pregel

- **Pregel** is a graph processing framework inspired by the BSP model
  - Pregel = Parallel + Graph + Google
  - Designed for large-scale distributed graph computations
  - Uses a vertex-centric programming model
    - Each vertex performs computation independently in supersteps
    - Motto: *"Think like a vertex"*
  - Vertices
    - Send messages to neighbors
    - Update state based on received messages
    - Can vote to halt; computation ends when all are inactive
- **PageRank example**
  - Each vertex divides its rank among neighbors
  - In each superstep, vertices update rank from incoming messages
- **Connected components example**
  - Each vertex propagates smallest ID seen to neighbors
  - Repeats until no changes occur

10 / 12

- **Pregel**
  - *Pregel* is a framework designed to handle graph processing tasks efficiently by drawing inspiration from the Bulk Synchronous Parallel (BSP) model. This model is particularly useful for managing large-scale distributed computations.
  - The name "Pregel" is a combination of "Parallel," "Graph," and "Google," indicating its purpose and origin.
  - **Vertex-Centric Programming Model**
    - In this model, each vertex in the graph operates independently, performing computations in a series of steps known as supersteps.
    - The guiding principle is to *"Think like a vertex,"* meaning that each vertex focuses on its own data and interactions with neighboring vertices.
  - **Vertices**
    - Vertices can send messages to their neighboring vertices, allowing them to share information and coordinate actions.
    - They update their state based on the messages they receive, which helps in processing and analyzing the graph.
    - Vertices have the ability to "vote to halt," meaning they can signal when they have completed their tasks. The computation concludes when all vertices are inactive.
- **PageRank Example**
  - In the PageRank algorithm, each vertex distributes its rank value among its neighbors, simulating the way web pages pass on their importance to linked pages.
  - During each superstep, vertices adjust their rank based on the messages they receive, which represent the rank contributions from their neighbors.
- **Connected Components Example**

- In this scenario, each vertex shares the smallest identifier (ID) it has encountered with its neighbors.
- This process continues until no further changes occur, effectively grouping vertices into connected components based on shared IDs.

11 / 12: Apache Giraph

Apache Giraph

- **Apache Giraph** is an open-source implementation of Pregel
  - Written in Java for high scalability
- Suitable for batch-processing large graphs
- **Integrates with Hadoop ecosystem**
  - Runs on Hadoop MapReduce
  - Uses HDFS for storage and YARN for resource management
- **Example use cases**:
  - Social network analysis (e.g., friend recommendation)
  - Web graph analysis (e.g., PageRank)
  - Biological network exploration (e.g., gene interaction)

11 / 12

- **Apache Giraph** is an open-source framework designed to handle large-scale graph processing. It is based on Google's Pregel model, which is specifically tailored for processing large graphs efficiently. Giraph is implemented in Java, which helps in achieving high scalability, making it suitable for processing vast amounts of data.
- **Batch-processing large graphs**: Giraph excels in scenarios where large graphs need to be processed in batches. This is particularly useful in applications where the entire graph needs to be analyzed or transformed at once.
- **Integration with the Hadoop ecosystem**: Giraph is designed to work seamlessly with Hadoop, leveraging its MapReduce framework for distributed data processing. It uses Hadoop Distributed File System (HDFS) for storage, ensuring data is easily accessible and manageable. Additionally, it utilizes YARN for resource management, which helps in efficiently allocating resources across the cluster.
- **Example use cases**:
  - *Social network analysis*: Giraph can be used to analyze social networks, such as recommending friends based on mutual connections or shared interests.
  - *Web graph analysis*: It is suitable for analyzing web graphs, like calculating PageRank, which is essential for search engine optimization.

– *Biological network exploration*: Giraph can explore complex biological networks, such as gene interactions, to uncover insights in bioinformatics research.

12 / 12: Apache Spark GraphX

Apache Spark GraphX

- **GraphX** is Spark's API for graph-parallel computation
- Seamless integration with Spark ecosystem and pipelines
- Graphs represented using RDDs
  - Vertices and edges as distributed collections
- Includes a Pregel API
  - Enables iterative message-passing computations
  - Preserves immutability and functional programming style
- **Strengths vs traditional graph DBs**
  - Scalable to very large graphs
  - Efficient for batch analytics and machine learning workflows
- **Limitations**
  - Less suitable for real-time queries and transactional workloads
  - Requires understanding of Spark internals for tuning performance

12 / 12

– **Apache Spark GraphX** is a powerful API designed for graph-parallel computation within the Spark ecosystem. It allows users to perform complex graph analytics seamlessly integrated with other Spark components, making it a versatile tool for data processing.

– **Integration with Spark**: GraphX fits naturally into the Spark ecosystem, allowing users to incorporate graph processing into their existing data pipelines. This integration leverages Spark's distributed computing capabilities, enhancing performance and scalability.

– **Graph Representation**: Graphs in GraphX are represented using Resilient Distributed Datasets (RDDs). This means that both vertices (nodes) and edges (connections) are stored as distributed collections, enabling efficient processing of large-scale graphs.

– **Pregel API**: GraphX includes a Pregel API, which supports iterative message-passing computations. This approach is useful for algorithms that require repeated updates, such as PageRank. It maintains immutability and adheres to a functional programming style, which is a hallmark of Spark.

– **Strengths**: Compared to traditional graph databases, GraphX is highly scalable, making it suitable for handling very large graphs. It is particularly efficient for batch analytics and machine learning workflows, where processing large datasets is crucial.

– **Limitations**: GraphX is less suited for real-time queries and transactional workloads, which are better handled by traditional graph databases. Additionally, optimizing performance in GraphX requires a good understanding of Spark internals, which can be a barrier for some

users.